

1.Producer & Consumer problem using Semaphores

Producer consumer problem is a classical synchronization problem. We can solve this problem by using semaphores.

A **semaphore** S is an integer variable that can be accessed only through two standard operations : wait() and signal().

The wait() operation reduces the value of semaphore by 1 and the signal() operation increases its value by 1.

```
wait(S){
while(S<=0); // busy waiting
S--;
}

signal(S){
S++;
}
```

Semaphores are of two types:

1. **Binary Semaphore** – This is also known as mutex lock. It can have only two values – 0 and 1. Its value is initialized to 1. It is used to implement solution of critical section problem with multiple processes.
2. **Counting Semaphore** – Its value can range over an unrestricted domain. It is used to control access to a resource that has multiple instances.

Problem Statement – We have a buffer of fixed size. A producer can produce an item and can place in the buffer. A consumer can pick items and can consume them. We need to ensure that when a producer is placing an item in the buffer, then at the same time consumer should not consume any item. In this problem, buffer is the critical section.

To solve this problem, we need two counting semaphores – Full and Empty. “Full” keeps track of number of items in the buffer at any given time and “Empty” keeps track of number of unoccupied slots.

Initialization of semaphores –

mutex = 1

Full = 0 // Initially, all slots are empty. Thus full slots are 0

Empty = n // All slots are empty initially

Solution for Producer –

```
do{
//produce an item
wait(empty);
wait(mutex);
//place in buffer
signal(mutex);
signal(full);
}while(true)
```

When producer produces an item then the value of “empty” is reduced by 1 because one slot will be filled now. The value of mutex is also reduced to prevent consumer to access the buffer. Now, the producer has placed the item and thus the value of “full” is increased by 1. The value of mutex is also increased by 1 because the task of producer has been completed and consumer can access the buffer.

Solution for Consumer –

```
do{
wait(full);
wait(mutex);
// remove item from buffer
signal(mutex);
signal(empty);
// consumes item
}while(true)
```

- As the consumer is removing an item from buffer, therefore the value of “full” is reduced by 1 and the value of mutex is also reduced so that the producer cannot access the buffer at this moment.

- Now, the consumer has consumed the item, thus increasing the value of “empty” by 1. The value of mutex is also increased so that producer can access the buffer now.

```
#include<stdio.h>
#include<stdlib.h>
int mutex=1,full=0,empty=3,x=0;
int main()
{
    int n;
    void producer();
    void consumer();
    int wait(int);
    int signal(int);
    printf("\n1.Producer\n2.Consumer\n3.Exit");
    while(1)
    {
        printf("\nEnter your choice:");
        scanf("%d",&n);
        switch(n)
        {
            case 1:  if((mutex==1)&&(empty!=0))
                      producer();
                    else
                      printf("Buffer is full!!");
                    break;
            case 2:  if((mutex==1)&&(full!=0))
                      consumer();
                    else
                      printf("Buffer is empty!!");
                    break;
            case 3:
                      exit(0);
                      break;
        }
    }

    return 0;
}

int wait(int s)
{
    return (--s);
}

int signal(int s)
{
    return(++s);
}

void producer()
{
    mutex=wait(mutex);
    full=signal(full);
    empty=wait(empty);
    x++;
    printf("\nProducer produces the item %d",x);
    mutex=signal(mutex);
}
```

```

}

void consumer()
{
    mutex=wait(mutex);
    full=wait(full);
    empty=signal(empty);
    printf("\nConsumer consumes item %d",x);
    x--;
    mutex=signal(mutex);
}

```

Output

```

1.Producer
2.Consumer
3.Exit
Enter your choice:1
Producer produces the item 1
Enter your choice:2
Consumer consumes item 1
Enter your choice:2
Buffer is empty!!
Enter your choice:1
Producer produces the item 1
Enter your choice:1
Producer produces the item 2
Enter your choice:1
Producer produces the item 3
Enter your choice:1
Buffer is full!!
Enter your choice:3

```

2. Write a program to implement producer consumer Shared memory

```

#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#define PERMS 0666
#define mkey 45787

```

file 2: sender.c

```

#include "shmem.h"
int main()
{
    int shmehid,n;
    char *pshmem;
    if((shmehid=shmget(mkey,10,PERMS|IPC_CREAT))<0)
    {
        printf("\n Sender: Cant get shared memory");
        exit(1);
    }
    pshmem=shmat(shmehid,(char *) 0,0);
    if(pshmem < 0)
    {

```

```
perror("Sender: Cant attach shared memory");
exit(1);
}
n=read(0,pshmem,10);
pshmem[n]='\0';
exit(0);
}
```

file 3: receiver.c

```
#include "shmem.h"
int main()
{
    int shmehid,n;
    char *pshmem;
    if((shmehid=shmget(mkey,10,PERMS|IPC_CREAT))<0)
    {
        printf("\nReceiver: Error opening shared memory");
        exit(1);
    }
    pshmem=shmat(shmehid,(char *) 0,0);
    if(pshmem < 0)
    {
        printf("\n Receiver : Can't attach shared memory");
        exit(1);
    }
    printf("%s",pshmem);
    shmctl(shmehid,IPC_RMID,NULL);
    exit(0);
}
```

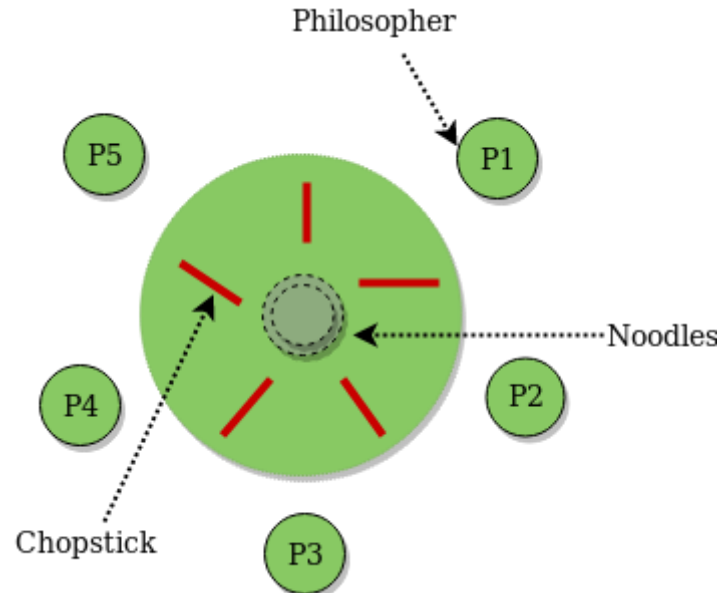
output

```
cc sender.c
$ ./a.out
hi
```

3. Dining Philosophers problem using Semaphores

Prerequisite – [Process Synchronization](#), [Semaphores](#), [Dining-Philosophers Solution Using Monitors](#)

The Dining Philosopher Problem – The Dining Philosopher Problem states that K philosophers seated around a circular table with one chopstick between each pair of philosophers. There is one chopstick between each philosopher. A philosopher may eat if he can pickup the two chopsticks adjacent to him. One chopstick may be picked up by any one of its adjacent followers but not both.



Semaphore Solution to Dining Philosopher –

Each philosopher is represented by the following pseudocode:

```
process P[i]
while true do
{ THINK;
  PICKUP(CHOPSTICK[i], CHOPSTICK[i+1 mod 5]);
  EAT;
  PUTDOWN(CHOPSTICK[i], CHOPSTICK[i+1 mod 5])
}
```

There are three states of philosopher : **THINKING, HUNGRY and EATING**. Here there are two semaphores : Mutex and a semaphore array for the philosophers. Mutex is used such that no two philosophers may access the pickup or putdown at the same time. The array is used to control the behavior of each philosopher. But, semaphores can result in deadlock due to programming errors.

```
#include<stdio.h>
#include<semaphore.h>
#include<pthread.h>

#define N 5
#define THINKING 0
#define HUNGRY 1
#define EATING 2
#define LEFT (ph_num+4)%N
#define RIGHT (ph_num+1)%N

sem_t mutex;
sem_t S[N];
```

```

void *philospher(void *num);
void take_fork(int);
void put_fork(int);
void test(int);

int state[N];
int phil_num[N]={0,1,2,3,4};

int main()
{
    int i;
    pthread_t thread_id[N];
    sem_init(&mutex,0,1);
    for(i=0;i<N;i++)
        sem_init(&S[i],0,0);
    for(i=0;i<N;i++)
    {
        pthread_create(&thread_id[i],NULL,philospher,&phil_num[i]);
        printf("Philosopher %d is thinking\n",i+1);
    }
    for(i=0;i<N;i++)
        pthread_join(thread_id[i],NULL);
}

void *philospher(void *num)
{
    while(1)
    {
        int *i = num;
        sleep(1);
        take_fork(*i);
        sleep(0);
        put_fork(*i);
    }
}

void take_fork(int ph_num)
{
    sem_wait(&mutex);
    state[ph_num] = HUNGRY;
    printf("Philosopher %d is Hungry\n",ph_num+1);
    test(ph_num);
    sem_post(&mutex);
    sem_wait(&S[ph_num]);
    sleep(1);
}

void test(int ph_num)
{
    if (state[ph_num] == HUNGRY && state[LEFT] != EATING && state[RIGHT] != EATING)
    {
        state[ph_num] = EATING;
        sleep(2);
        printf("Philosopher %d takes fork %d and %d\n",ph_num+1,LEFT+1,ph_num+1);
        printf("Philosopher %d is Eating\n",ph_num+1);
        sem_post(&S[ph_num]);
    }
}

void put_fork(int ph_num)

```

```

{
    sem_wait(&mutex);
    state[ph_num] = THINKING;
    printf("Philosopher %d putting fork %d and %d down\n",ph_num+1,LEFT+1,ph_num+1);
    printf("Philosopher %d is thinking\n",ph_num+1);
    test(LEFT);
    test(RIGHT);
    sem_post(&mutex);
}

```

4. Readers and Writers problem using Semaphores

Consider a situation where we have a file shared between many people.

- If one of the people tries editing the file, no other person should be reading or writing at the same time, otherwise changes will not be visible to him/her.
- However if some person is reading the file, then others may read it at the same time.

Precisely in OS we call this situation as the **readers-writers problem**

Problem parameters:

- One set of data is shared among a number of processes
- Once a writer is ready, it performs its write. Only one writer may write at a time
- If a process is writing, no other process can read it
- If at least one reader is reading, no other process can write
- Readers may not write and only read

Solution when Reader has the Priority over Writer

Here priority means, no reader should wait if the share is currently opened for reading.

Three variables are used: **mutex**, **wrt**, **readcnt** to implement solution

1. **semaphore** mutex, wrt; // semaphore **mutex** is used to ensure mutual exclusion when **readcnt** is updated i.e. when any reader enters or exit from the critical section and semaphore **wrt** is used by both readers and writers
2. **int** readcnt; // **readcnt** tells the number of processes performing read in the critical section, initially 0

Functions for semaphore :

– wait() : decrements the semaphore value.

– signal() : increments the semaphore value.

Writer process:

1. Writer requests the entry to critical section.
2. If allowed i.e. wait() gives a true value, it enters and performs the write. If not allowed, it keeps on waiting.
3. It exits the critical section.

```

do {
    // writer requests for critical section
    wait(wrt);
    // performs the write
    // leaves the critical section
    signal(wrt);
} while(true);

```

Reader process:

1. Reader requests the entry to critical section.

2. If allowed:
 - it increments the count of number of readers inside the critical section. If this reader is the first reader entering, it locks the **wrt** semaphore to restrict the entry of writers if any reader is inside.
 - It then, signals mutex as any other reader is allowed to enter while others are already reading.
 - After performing reading, it exits the critical section. When exiting, it checks if no more reader is inside, it signals the semaphore “wrt” as now, writer can enter the critical section.
3. If not allowed, it keeps on waiting.

```
do {  
    // Reader wants to enter the critical section  
    wait(mutex);  
    // The number of readers has now increased by 1  
    readcnt++;  
    // there is atleast one reader in the critical section  
    // this ensure no writer can enter if there is even one reader  
    // thus we give preference to readers here  
    if (readcnt==1)  
        wait(wrt);  
  
    // other readers can enter while this current reader is inside  
    // the critical section  
    signal(mutex);  
  
    // current reader performs reading here  
    wait(mutex); // a reader wants to leave  
  
    readcnt--;  
  
    // that is, no reader is left in the critical section,  
    if (readcnt == 0)  
        signal(wrt); // writers can enter  
  
    signal(mutex); // reader leaves  
} while(true);
```

Thus, the semaphore ‘**wrt**’ is queued on both readers and writers in a manner such that preference is given to readers if writers are also there. Thus, no reader is waiting simply because a writer has requested to enter the critical section.

